

Домашня робота 3: 2D бінарне неперервне середовище

Агент, сприйняття та комп'ютерний зір у динамічному
клітинно-автоматному світі

2026-03-05

Зміст

1 Вступ	2
2 Середовище (World)	2
2.1 Стан та ініціалізація	2
2.2 Динаміка: «Гра Життя»	3
2.3 Візуалізація	4
3 Агент (Agent)	4
3.1 Неперервна позиція та фізика	4
3.2 Модель зіткнення	4
4 Сприйняття: ортографічна камера	5
4.1 Принцип проекції	5
4.2 Режими фільтрації (текстурна фільтрація)	5
5 Комп'ютерний зір (FeatureTracker)	6
5.1 Структура даних: Feature	6
5.2 Детектор кутових точок Харріса	7
5.2.1 Параметри та їх вплив	7
5.2.2 NMS (Non-Maximum Suppression)	7
5.3 Прив'язка треків між кадрами	7
5.4 Зворотна проекція (ego → world)	8
6 Модель руху (TrajectoryBuffer)	8

7 Загальна архітектура	8
8 Завдання для студентів	9
8.1 Задача 1 — Покращення детектора Харріса	9
8.2 Задача 2 — Перехід від егоцентричних до алоцентричних координат	10
8.3 Додаткові завдання (за бажанням)	11
8.3.1 Задача 3 — Оптичний потік	11
8.3.2 Задача 4 — Предикція положення агента	11
8.3.3 Задача 5 — Автономна навігація	11
9 Інструкція з запуску	11
10 Список літератури	12

1. Вступ

У цій роботі розглядається класична задача AGI-досліджень: **автономний агент, занурений у динамічне середовище**. Середовище реалізоване як двовимірний клітинний автомат за правилами «Гри Життя» Конвея; агент існує у ньому як фізичне тіло з неперервними координатами, власною динамікою та обмеженим каналом сприйняття — ортографічною камерою.

Мета — дослідити три рівні взаємодії:

1. **Середовище** — автономна динаміка, незалежна від агента.
2. **Агент** — фізика руху, зіткнення, керування.
3. **Сприйняття** — відображення стану світу у простір вимірювань агента; виявлення та відстеження ознак.

2. Середовище (World)

2.1. Стан та ініціалізація

Світ — це тор розміром $W \times H$ клітин. Кожна клітина бінарна: **1** (жива, біла) або **0** (мертва, чорна). Початковий стан — випадковий рівномірний розподіл.

```
class World:
    def __init__(self, width=100, height=100):
        self.width = width
        self.height = height
        self.state = np.random.randint(0, 2, size=(height, width))
        self.time_step = 0
```

2.2. Динаміка: «Гра Життя»

На кожному кроці застосовуються правила Конвея:

Стан клітини	Кількість живих сусідів	Новий стан
жива	2 або 3	жива (виживає)
жива	інше	мертва (гине)
мертва	рівно 3	жива (народжується)
мертва	інше	мертва

Реалізація використовує **єдиний прохід згортки через np.pad** замість 8 окремих np.roll — це зменшує кількість тимчасових масивів і прискорює обчислення.

```
def step(self):
    p = np.pad(self.state, 1, mode='wrap') # тороїдальні межі
    neighbours = (p[:-2, :-2] + p[:-2, 1:-1] + p[:-2, 2:] +
                  p[1:-1, :-2] + p[1:-1, 2:] +
                  p[2:, :-2] + p[2:, 1:-1] + p[2:, 2:])
    survive = (self.state == 1) & ((neighbours == 2) | (neighbours == 3))
    born = (self.state == 0) & (neighbours == 3)
    self.state = (survive | born).view(np.uint8)
    self.time_step += 1
```

Важливо: після достатньої кількості кроків (~400-1000 залежно від початкового стану) «Гра Життя» досягає характерного режиму з рухомими структурами (планерами, осциляторами тощо). Саме цей режим є дослідницьки цікавим — середовище динамічне, але структуроване.

2.3. Візуалізація

`World.visualize_opencv()` будує зображення без Python-циклів через `np.repeat` та малює:

- **Білі клітини** — живі; **чорні** — мертві.
 - **Червоне коло** — агент (неперервна позиція).
 - **Зелений прямокутник** — поле зору камери (з тороїдальним розбиттям на до 4 сегментів).
-

3. Агент (Agent)

3.1. Неперервна позиція та фізика

Позиція агента — пара дійсних чисел $(x, y) \in [0, W) \times [0, H)$. Рух описується **імпульсною динамікою** (аналог 1D неперервного випадку з практики 1):

$$v_i \leftarrow v_i + \frac{F_{\text{kick}}}{m}, \quad v_i \leftarrow v_i - \frac{\mu}{m} v_i \Delta t, \quad x_i \leftarrow x_i + v_i \Delta t$$

де μ — коефіцієнт тертя, m — маса агента, F_{kick} — миттєвий імпульс від команди керування.

```
# Параметри за замовчуванням
self.mass = 1.0
self.friction_coeff = 5.0 # швидке затухання
self.kick_force = 50.0 # сила поштовху
```

3.2. Модель зіткнення

Перед кожним кроком інтеграції кандидатна позиція перевіряється на відповідну клітину світу. Якщо клітина жива — відбувається **пружний відбій** зі згасанням:

$$v_i \leftarrow -e \cdot v_i, \quad e = 0.45$$

Осі x та y обробляються незалежно — агент ковзає вздовж стін при діагональному русі.

```

def update(self, dt, restitution=0.45):
    for i in range(2):
        # тертя
        self.velocity[i] += (-self.friction_coeff * self.velocity[i] / self.mass) * dt
        dim = self.world.width if i == 0 else self.world.height
        new_coord = (self.position[i] + self.velocity[i] * dt) % dim
        # перевірка зіткнення
        cx = int(new_coord) % self.world.width if i == 0 else int(self.position[0]) % self.world.width
        cy = int(self.position[1]) % self.world.height if i == 0 else int(new_coord) % self.world.height
        if self.world.state[cy, cx] == 1:
            self.velocity[i] = -self.velocity[i] * restitution # відбій
        else:
            self.position[i] = new_coord

```

4. Сприйняття: ортографічна камера

4.1. Принцип проєкції

Камера агента — це відображення прямокутної ділянки світу розміром $\text{fov}_w \times \text{fov}_h$ (у одиницях клітин) на пікселів-матрицю $C_w \times C_h$. Для кожного пікселя (p_x, p_y) виконується:

$$w_x = x_{\text{agent}} + \left(\frac{p_x}{C_w} - 0.5 \right) \cdot \text{fov}_w$$

$$w_y = y_{\text{agent}} + \left(\frac{p_y}{C_h} - 0.5 \right) \cdot \text{fov}_h$$

Отримані w_x, w_y — координати у просторі світу (з тороїдальним загортанням).

4.2. Режими фільтрації (текстурна фільтрація)

Режим	Сірі пікселі	Аналог у графіці
nearest	ні	Minecraft / піксель-арт
bilinear	так, плавно	Стандартний фільтр OpenGL

Режим	Сірі пікселі	Аналог у графіці
area	так, точно	Мімап / матриця сенсора

Найближчий сусід (nearest):

$$I[p_y, p_x] = \text{state}[\lfloor w_y \rfloor \bmod H, \lfloor w_x \rfloor \bmod W]$$

Білінійна інтерполяція (bilinear) — зважена сума 4 сусідніх клітин:

$$I = c_{00}(1 - f_x)(1 - f_y) + c_{10}f_x(1 - f_y) + c_{01}(1 - f_x)f_y + c_{11}f_xf_y$$

де $f_x = w_x - \lfloor w_x \rfloor$, $f_y = w_y - \lfloor w_y \rfloor$ — дробові частини.

Area / box-filter (area) — усереднення 4×4 стратифікованих підвибірок у «footprint» кожного пікселя. Природна анти-аліасинг-фільтрація при великому fov.

Примітка: перед застосуванням фільтра Харріса зображення додатково розмивається гауссіаном ($\sigma = 1.5$), щоб надати похідним Собеля гладкі перепади на бінарних краях.

5. Комп'ютерний зір (FeatureTracker)

5.1. Структура даних: Feature

```
@dataclass
class Feature:
    id: int # унікальний персистентний ідентифікатор треку
    px: float # колонка у пікселях камери
    py: float # рядок у пікселях камери
    strength: float # відгук детектора Харріса
    world_x: float # зворотна проекція → координата x у світі
    world_y: float # зворотна проекція → координата y у світі
    age: int # кількість кадрів, протягом яких трек активний
```

5.2. Детектор кутових точок Харріса

Детектор Харріса шукає точки, де **матриця других моментів** M має два великі власні значення:

$$M = \sum_{(x,y) \in W} \begin{pmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{pmatrix}, \quad R = \det(M) - k \cdot \text{tr}(M)^2$$

Точки з $R > \theta \cdot R_{\max}$ вважаються кутами.

5.2.1. Параметри та їх вплив

Параметр	Значення	Роль
block_size	3	Розмір вікна інтеграції (менше → точніша локалізація)
ksize	3	Апертура Собеля
harris_k	0.04	Чутливість (стандартне значення)
threshold	0.005	Частка від $R_{\max}$; нижче → більше точок
blur_sigma	1.5	Гауссів розмив перед детекцією

5.2.2. NMS (Non-Maximum Suppression)

Після порогування залишаються лише **найсильніші** кути, причому між будь-якими двома збереженими точками відстань $\geq \text{min_distance}$ (жадібний алгоритм по спадній силі відгуку).

5.3. Прив'язка треків між кадрами

Оскільки середовище динамічне (клітини народжуються та гинуть), кути можуть зникати й з'являтися кожен кадр. Використовується **найближчий сусід у просторі зображення**:

для кожного нового виявлення (px, py):
 знайти найближчий попередній трек
 якщо відстань $\leq$ match_radius $\rightarrow$ успадкувати id та збільшити age
 інакше $\rightarrow$ новий id, age = 1

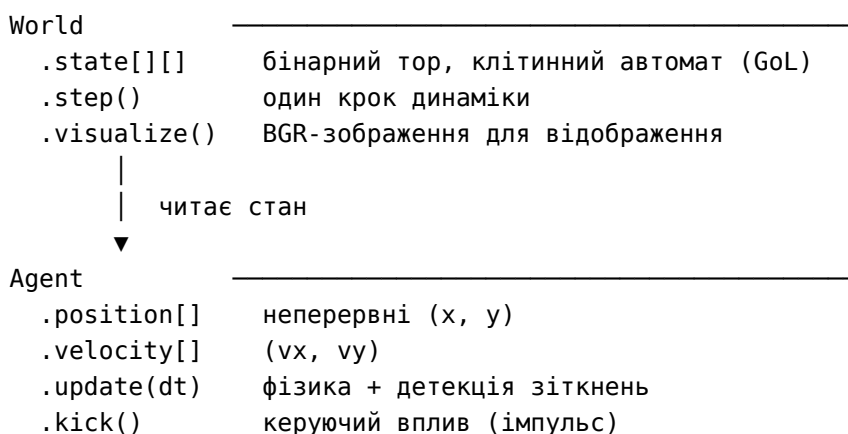
5.4. Зворотна проєкція (ego $\rightarrow$ world)

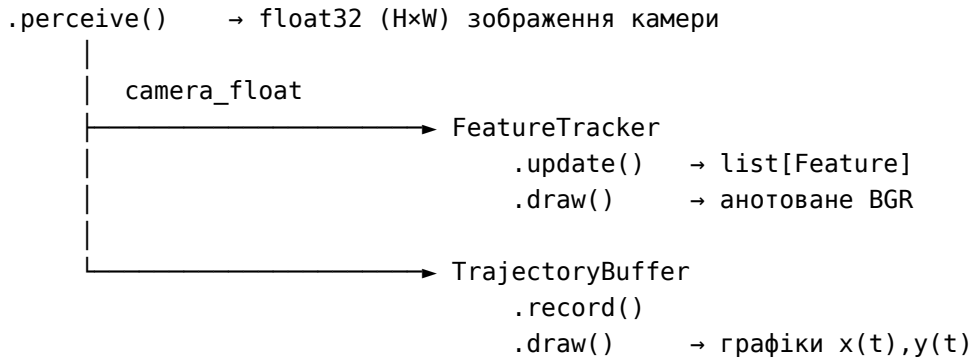
```
def _backproject(self, px, py, agent):
    wx = agent.position[0] + (px / agent.camera_w - 0.5) * agent.fov_w
    wy = agent.position[1] + (py / agent.camera_h - 0.5) * agent.fov_h
    return wx % agent.world.width, wy % agent.world.height
```

6. Модель руху (TrajectoryBuffer)

Клас TrajectoryBuffer зберігає ковзне вікно з $N_{\max}$ відліків (t, x, y) та рендерить два графіки у реальному часі — $x(t)$ та $y(t)$ — у власне OpenCV-зображення без залежності від matplotlib. Поточна позиція позначається білою крапкою з числовим підписом.

7. Загальна архітектура





8. Завдання для студентів

8.1. Задача 1 — Покращення детектора Харріса

Проблема: поточна реалізація знаходить кути у координатах пікселів зображення камери, але внаслідок дискретизації координати кутів зміщені відносно справжніх субпіксельних положень.

Завдання: реалізувати **субпіксельне уточнення** положень кутів.

OpenCV надає готову функцію:

```

cv2.cornerSubPix(
    image,
    corners,          # (N, 1, 2) float32
    winSize=(5, 5),  # вікно пошуку
    zeroZone=(-1, -1),
    criteria=(cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 30, 0.001)
)
  
```

Очікуваний результат:

- координати `Feature.px`, `Feature.py` — дійсні числа з точністю до частки пікселя;
- зворотна проекція у `world_x`, `world_y` стає точнішою відповідно;
- візуально: кола на Camera View точно центровані на кутах об'єктів.

Підказки:

1. Викликати `cornerSubPix` після NMS, передавши масив знайдених точок.

2. Зображення для уточнення має бути тим самим `img_f` (після Гауссового розмиву), що й для детекції.
3. Порівняйте візуально результат зі і без уточнення при різних режимах фільтрації (nearest vs bilinear).

8.2. Задача 2 — Перехід від егоцентричних до алоцентричних координат

Контекст: кожна ознака `Feature` зберігає `world_x`, `world_y` — координати у системі відліку **світу** (алоцентричні). Проте відстані та кути між ознаками поки не обчислюються.

Завдання: для кожної пари ознак (i, j) у поточному кадрі обчислити **відносні координати** ознаки j у системі відліку ознаки i :

$$\Delta x_{ij} = x_j - x_i, \quad \Delta y_{ij} = y_j - y_i$$
$$d_{ij} = \sqrt{\Delta x^2 + \Delta y^2}, \quad \theta_{ij} = \text{atan2}(\Delta y_{ij}, \Delta x_{ij})$$

з урахуванням **тороїдального загортання** простору (найкоротший шлях на торі).

Завдання-мінімум:

- Додати до `FeatureTracker` метод `relative_descriptors()`, що повертає словник `{(id_i, id_j): (dx, dy, dist, angle)}` для всіх пар.
- Намалювати на `Camera View` лінії між ознаками, що існують ≥ 5 кадрів, кольором, пропорційним відстані.

Завдання-максимум:

- Розглянути **граф ознак**: вузли — кути, ребра — пари у радіусі R пікселів. Чи стабільний цей граф між кадрами?

8.3. Додаткові завдання (за бажанням)

8.3.1. Задача 3 — Оптичний потік

Реалізувати відстеження методом **Lucas-Kanade** (замість або поряд із NN-matching):

```
next_pts, status, err = cv2.calcOpticalFlowPyrLK(
    prev_grey, curr_grey, prev_pts, None,
    winSize=(15, 15), maxLevel=2,
    criteria=(cv2.TERM_CRITERIA_EPS | cv2.TERM_CRITERIA_COUNT, 10, 0.03)
)
```

Порівняйте стабільність треків (метрика: середнє age активних треків) між NN-matching та LK-flow.

8.3.2. Задача 4 — Предикція положення агента

Використовуючи `TrajectoryBuffer`, побудувати простий **фільтр Калмана** (стан: (x, y, v_x, v_y) , спостереження: (x, y)) і порівняти відфільтровану траєкторію зі сирими даними. Коли фільтр найефективніший — при зупиненому агенті, при рівномірному русі чи при зіткненнях?

8.3.3. Задача 5 — Автономна навігація

Реалізувати простий **рефлекторний агент**: якщо у центральній чверті кадру камери немає живих клітин — застосувати випадковий поштовх; якщо є — відвернутись (поштовх у напрямку меншої щільності ознак). Дослідіть, чи виникає стійка поведінка.

9. Інструкція з запуску

```
# 1. Встановити залежності
pip install numpy opencv-python

# 2. Запустити симуляцію
python practice_3/2d_binary_continuous.py
```

Клавіша	Дія
w / s / a / d	Поштовх вгору / вниз / ліворуч / праворуч
Space	Пауза / продовження GoL
+ / -	Швидше / повільніше GoL
p	Цикл режимів камери: bilinear → area → nearest
f	Увімк./вимк. overlay кутових точок
c	Очистити буфер траєкторії
e / z / x	Перемикнути / вбити / відродити клітину під агентом
q або Esc	Вихід

10. Список літератури

- Conway, J.H. (1970). The Game of Life. Scientific American.
- Harris, C., Stephens, M. (1988). A Combined Corner and Edge Detector. Proc. Alvey Vision Conference.
- Shi, J., Tomasi, C. (1994). Good Features to Track. CVPR.
- Bradski, G., Kaehler, A. (2008). Learning OpenCV. O'Reilly.
- Kalman, R.E. (1960). A New Approach to Linear Filtering and Prediction Problems. ASME Journal.